

OpenZeppelin Confidential Contracts v0.5 Diff Audit

The ZAMA logo is displayed in a large, bold, black sans-serif font, centered within a solid yellow rectangular background.

June 15, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Hook Module Framework	5
Account Recovery on ERC7984Rwa	5
Identity Check Extension	6
Core Transfer Semantics	6
FHESafeMath Saturating Primitives	6
BatcherConfidential Distinct-Underlying Invariant	6
Security Model and Trust Assumptions	7
Privileged Roles	9
Medium Severity	10
M-01 Self-From Transfer Entrypoints Allow Permissionless Zero-Amount Transfers via Empty-Proof Shortcut	10
M-02 Dispatched Batches Have No Time-Bounded Recovery Path When Finalization or Cancellation Reverts	11
M-03 ERC-2771 Meta-Transactions Make the Forwarder the Ciphertext Principal in ERC7984Rwa	13
M-04 Concurrent Partial Batches Can Misallocate Target-Underlying Output Across Batches	14
M-05 Operator-Held Transient Access Can Be Escalated to Permanent Public Disclosure of Transferred Amounts	15
Notes & Additional Information	17
N-01 confidentialAvailable Grants Persistent ACL Without Validating Result or Caller	17
N-02 Unlimited Underlying Approvals Expose Batcher-Held Funds to Wrapper Compromise	18
N-03 onUninstall Lacks Installed-Module Privileges and Reverts Silently	19
N-04 Hardcoded Selector Literals in _isForceTransfer Are Silent-Failure-Prone	20
N-05 Code Quality and Documentation	21
N-06 Installed Hook Modules Can Make Unbounded FHE ACL Grants That Survive Uninstall	22
Client Reported	23
CR-01 Trivial-Case Early Return in ERC7984HolderCapHookModule._preTransfer Skips the super Call	23
Conclusion	25
Appendix	27
Issue Classification	27

Summary

Type	Library	Total Issues	12 (6 resolved)
Timeline	From 2026-05-18 To 2026-05-21	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	5 (2 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	6 (3 resolved)
		Client Reported Issues	1 (1 resolved)

Scope

OpenZeppelin performed a diff audit of the [OpenZeppelin/openzeppelin-confidential-contracts](#) repository at the [release-v0.5](#) head commit [49e71451](#) against the previously audited [release-v0.4](#) head commit [c0274695](#) with [this diff](#).

In scope were the following files:

```
contracts
├── finance
│   └── BatchConfidential.sol (diff)
├── interfaces
│   ├── IERC7984HookModule.sol (new)
│   └── IERC7984Rwa.sol (diff)
├── token
│   ├── ERC7984
│   │   ├── ERC7984.sol (diff)
│   │   ├── extensions
│   │   │   ├── ERC7984Freezable.sol (diff)
│   │   │   ├── ERC7984Hooked.sol (new)
│   │   │   ├── ERC7984IdentityCheck.sol (new)
│   │   │   ├── ERC7984Restricted.sol (diff)
│   │   │   └── ERC7984Rwa.sol (diff)
│   │   └── utils
│   │       ├── ERC7984BalanceCapHookModule.sol (new)
│   │       ├── ERC7984HolderCapHookModule.sol (new)
│   │       └── ERC7984HookModule.sol (new)
│   └── utils
│       ├── FHESafeMath.sol (diff)
│       └── HandleAccessManager.sol (diff)
```

Update: The fixes for the findings highlighted in this report have all been merged at commit [0c41888](#).

System Overview

OpenZeppelin Confidential Contracts is a Solidity library of confidential token primitives built on Zama's [@fhevm/solidity](#) FHE runtime. Balances, allowances, and per-account compliance state are stored as encrypted handles; control flow that depends on values is expressed through branchless FHE operations such as [FHE.select](#). The library extends the OpenZeppelin Contracts architecture to support encrypted token balances and transfers, where sensitive financial data is processed as ciphertexts that are never exposed on-chain in plaintext without an explicit, gated decryption step. The [v0.5](#) release candidate extends the previously audited [v0.4](#) base in five directions.

Hook Module Framework

[ERC7984Hooked](#) is a new extension that runs installed hook modules before and after every [_update](#). Modules implement [IERC7984HookModule](#) ([preTransfer](#), [postTransfer](#), [onInstall](#), [onUninstall](#)) and are installed under access control delegated to the concrete contract through [_authorizeModuleChange](#). The pre-hook aggregates each module's [ebool](#) compliance result and [FHE.select](#)s the transfer amount to zero when any module disapproves. The post-hook notifies modules of the final transferred amount for downstream accounting. The release ships an abstract base ([ERC7984HookModule](#)) plus two example modules: [ERC7984BalanceCapHookModule](#) for a per-investor encrypted balance cap and [ERC7984HolderCapHookModule](#) for a per-token encrypted holder-count cap. The framework is inspired by ERC-7579 modules, but uses a single hook type rather than the four ERC-7579 module types.

Account Recovery on [ERC7984Rwa](#)

[recoverAddress\(lostAccount, newAccount\)](#) is a new agent-only operation that migrates an account's full balance and frozen-balance state to a replacement address, propagating the [BLOCKED](#) restriction when applicable. The flow uses the new [FHESafeMath](#) saturating primitives to redistribute the encrypted frozen amount, and the operation's selector is added to the existing [_isForceTransfer](#) set so pause and sender-restriction checks are bypassed during the underlying transfer. Force-transfer plumbing was refactored in the same release to inline what was previously a [_forceUpdate](#) helper, with no semantic change.

Identity Check Extension

`ERC7984IdentityCheck` overrides `_update` to require that any non-zero recipient be verified by an external `IIdentityRegistry`. The pattern follows the ERC-3643 (T-REX) reference, gating on the recipient only. The identity registry is set at construction; the concrete contract is responsible for exposing an access-controlled setter.

Core Transfer Semantics

`ERC7984._update` no longer reverts when invoked with an uninitialized `fromBalance` (the `ERC7984ZeroBalance` revert was removed). Transfers from never-funded accounts now fall through to a zero-amount transfer via `FHESafeMath.tryDecrease`, which returns initialized handles even when the source balance handle is uninitialized. Additionally, `_transferAndCall` was refactored so the transient ACL grant on the returned ciphertext now happens inside the helper rather than in each public wrapper.

FHESafeMath Saturating Primitives

`saturatingAdd` and `saturatingSub` are new helpers that clamp at `type(uint64).max` and zero respectively. `saturatingSub` is consumed in `ERC7984Freezable._confidentialAvailable` (replacing a prior `tryDecrease + FHE.select` pattern) and in `ERC7984Rwa.recoverAddress`. `saturatingAdd` is consumed in `recoverAddress` for frozen-balance preservation.

BatcherConfidential Distinct-Underlying Invariant

`BatcherConfidential`'s constructor now rejects two wrappers that share the same `underlying()` token, preventing nonsensical self-arbitrage configurations.

Security Model and Trust Assumptions

The audit was performed under a standard OpenZeppelin trust model with adjustments specific to this engagement. Findings that require a privileged role to act maliciously have been discarded except where explicitly noted.

The following trust assumptions apply throughout:

- **AGENT_ROLE** in `ERC7984Rwa` is honest and competent. The role can mint, burn, force-transfer, recover any address, set frozen amounts, pause the contract, and block users. Compromise of the agent key is treated as out-of-scope; key management is the deployer's responsibility.
- **DEFAULT_ADMIN_ROLE** is honest within the audit scope. The role is responsible for assigning **AGENT_ROLE** and is set at construction with no timelock. Deployments are expected to back this role with a multi-signature wallet or timelock in production.
- `_authorizeModuleChange` in `ERC7984Hooked` is correctly implemented by the concrete deploying contract. The framework leaves the access-control predicate abstract; deployers must gate module installation at the admin tier.
- `@fhevm/solidity` FHE runtime is treated as a trusted black box. Findings must identify misuse of FHE primitives or library-documented semantics, not bugs inside the library itself.
- **IIIdentityRegistry** (the ERC-3643-style external contract consumed by `ERC7984IdentityCheck`) is examined as an integration boundary. The registry may revert, return arbitrary booleans, or change behavior across calls.
- **Hook modules** are confidentiality-trusted and compositional-correctness-trusted per the `class natspec`: an installed module receives FHE handle access via `_validateHandleAllowance`, and trust extends beyond read access because installed modules can reenter the token during `preTransfer` and mutate state used by other modules' checks. Deployments using multiple modules treat installation as a trust decision over the entire installed set. The audit examined the two in-scope example modules and the trust mechanisms by which a malicious or buggy third-party module could compromise token invariants.
- **The module set is stable during a single transfer.** Pre-hook and post-hook loops re-read `modules()` independently, so a hook with admin authority that reenters into `installModule` or `uninstallModule` can diverge the pre and post iterations. The

audit assumes module-mutation calls do not occur within a transfer's hook execution window.

- **IERC7984ERC20Wrapper** is trusted to return a canonical underlying-token address through `underlying()` and is a structurally trusted dependency of **BatcherConfidential**, which grants unlimited approval at construction and depends on the wrapper for correctness of `unwrap`, `finalizeUnwrap`, and `wrap`. The canonical **IERC7984ERC20Wrapper** is non-upgradeable; deployers extending the wrapper with upgradeability inherit broader trust exposure. The wrapper contracts themselves are out of scope.
- **FHE decryption gateway** is the external authority for `BatcherConfidential.dispatchBatchCallback`. The gateway-signed decryption proof is the authentication primitive.
- **transferAndCall** refund legs pass through the full `_update` policy chain. The refund `_update(to, from, ...)` always runs, even when the encrypted amount is zero. Recipient-side guards in **ERC7984IdentityCheck**, **ERC7984Restricted**, and the pause modifier fire on the original sender. Encrypted `success` prevents cheap branching to skip the refund.
- **setOperator** is not gated by restriction or pause. Block and pause are containment primitives over transfers, not approval state. An attacker controlling a compromised key can grant operator authority during a block and have that authority remain usable after the account is unblocked.
- **confidentialAvailable** produces persistent ACL entries by design, as required for the off-chain decryption UX (verified by the `test mock's confidentialAvailableAccess helper` passing `persistent=true`). Any caller can fund persistent ACL growth for arbitrary `account` arguments; the grant target is the rightful holder, so there is no information leak to the caller.
- **BatcherConfidential** inherits `fromToken` and `toToken` recipient policy for refunds and claims. `quit()` refunds to `msg.sender` and `claim()` sends to the original depositor; both honor the underlying token's recipient policy. If the recipient becomes permanently barred from receiving the token, the deposit or claim is operationally stranded with no batcher-level override.
- **Recovery propagates restrictive state (BLOCKED) only as a security exception** (subject to confirmation with the development team). `recoverAddress` migrates balance and frozen state and propagates **BLOCKED** to prevent block-evasion. **ALLOWED**, operator state, and `lostAccount` retirement are agent operational responsibility, not in-contract behavior.

The audit scope explicitly excluded the following files, either by client request or by default scope rules: tests, mocks, documentation, changesets, build and CI configuration, and the

eight production contracts that received no in-scope changes (`IERC7984ERC20Wrapper` , `IERC7984Receiver` , `ERC7984ERC20Wrapper` , `ERC7984ObserverAccess` , `ERC7984Votes` , `CheckpointsConfidential` , `VestingWalletConfidential` , `VotesConfidential`).

Privileged Roles

`ERC7984Rwa` defines two roles via OpenZeppelin `AccessControl` . The `DEFAULT_ADMIN_ROLE` is granted to the address passed to the constructor and is responsible for assigning and revoking the `AGENT_ROLE` . Accounts holding `AGENT_ROLE` have broad operational capabilities: minting and burning tokens, pausing and unpausing the contract, blocking and unblocking accounts, setting confidential frozen balances, recovering compromised addresses, and executing force transfers that bypass pause and sender-restriction checks. The security of the system depends on the integrity of these role holders.

`ERC7984Hooked` exposes module installation and uninstallation through the abstract `_authorizeModuleChange` predicate. The framework leaves the access-control predicate unimplemented; the concrete deploying contract is expected to gate these operations at the admin tier, given that module installation is an irrevocable confidentiality decision over the token's encrypted state.

`BatcherConfidential` defines no privileged roles. `dispatchBatch` and `dispatchBatchCallback` are permissionless by default; access restrictions are left to the implementing contract.

Medium Severity

M-01 Self-From Transfer Entrypoints Allow Permissionless Zero-Amount Transfers via Empty-Proof Shortcut

[ERC7984](#) exposes four public transfer entrypoints that accept an `externalEuint64` ciphertext handle plus a separate `inputProof`: the self-from variants `confidentialTransfer` and `confidentialTransferAndCall`, and the operator-gated `confidentialTransferFrom` and `confidentialTransferFromAndCall` variants. Each ingests the ciphertext via `FHE.fromExternal(encryptedAmount, inputProof)`, which is intended to bind the caller to the amount through proof verification.

In `@fhevm/solidity@0.11.1`, `FHE.fromExternal` admits a special-case path: when `inputProof.length == 0` and the input handle is `bytes32(0)`, it returns `asEuint64(0)`, a freshly trivially encrypted zero, without verifying any proof and without requiring any prior ACL. The optimization exists to support re-importing a previously allowed non-zero handle, but in the zero-handle sub-case it lets any caller invoke `confidentialTransfer(victim, externalEuint64.wrap(0), bytes(""))` or its `AndCall` sibling without holding any tokens, owning any FHE allowance, or interacting with the KMS. The operator-gated `From` variants are not reachable for arbitrary `from` because of the `isOperator` check; the self-from variants have no equivalent gate.

Each such call executes the full `_update` chain across every override in the inheritance graph: sender and recipient restriction checks in `ERC7984Restricted._update`, frozen-state reads in `ERC7984Freezable._update`, pre-hook and post-hook invocations on every installed module in `ERC7984Hooked._update` with the attacker-supplied `(from, to)` pair, and the external `IIdentityRegistry.isVerified(to)` call in `ERC7984IdentityCheck._update`. The call rewrites `_balances[from]` and `_balances[to]` with re-encrypted ciphertexts, issues fresh persistent FHE ACL grants on the new handles, and emits a `ConfidentialTransfer` event with an encrypted-zero amount. For `AndCall` variants whose receiver implements `IERC7984Receiver`, the callback `onConfidentialTransferReceived(operator, from, encrypted_zero, data)` runs with attacker-supplied `data`, followed by a refund `_update`.

A concrete reachable target in scope is

`BatcherConfidential.onConfidentialTransferReceived`. An attacker calling `fromToken.confidentialTransferAndCall(batcherAddress, externalEuint64.wrap(0), bytes(""), bytes(""))` satisfies the `msg.sender == address(fromToken())` check and triggers `_join(attacker, encrypted_zero)`, permanently initializing a zero-deposit entry in `_batches[batchId].deposits[attacker]` that no standard flow can remove. Repeating across batches bloats Batcher storage with attacker-controlled entries.

Consider rejecting the zero-handle case at each `externalEuint64`-accepting transfer entrypoint by requiring the unwrapped handle to be non-zero (`require(externalEuint64.unwrap(encryptedAmount) != bytes32(0), ERC7984InvalidZeroHandle())`), with a corresponding custom-error declaration. The guard closes the empty-proof bypass without breaking the legitimate optimization that permits an empty `inputProof` against a previously allowed non-zero handle. Consider applying the same guard to every other public function in the protocol that ingests `externalEuint64`, so the bypass is closed uniformly across the public surface. Consider also documenting in the NatSpec that zero-handle inputs are invalid, and recommending that receiver implementations of `IERC7984Receiver` validate the `amount` parameter rather than treating any callback as evidence of a non-zero transfer.

Update: Acknowledged, not resolved. The OpenZeppelin development team stated:

Transferring 0 amounts is a feature of the ERC7984 implementation. We do not intend to limit this in any way.

M-02 Dispatched Batches Have No Time-Bounded Recovery Path When Finalization or Cancellation Reverts

The `BatcherConfidential` state machine moves a batch into `Dispatched` when `dispatchBatch` creates an unwrap request, and the only exits from `Dispatched` are through `dispatchBatchCallback`, which must either set `exchangeRate` in the `Complete branch` or set `canceled = true` in the `Cancel branch`. Depositor exit via `quit` is gated to `Pending` and `Canceled`, and `claim` is gated to `Finalized`, so a batch trapped in `Dispatched` blocks both withdrawal paths.

Several revert sites along the callback can leave the batch stranded. The `Complete` branch computes `swappedAmount =`

`IERC20(toToken().underlying()).balanceOf(address(this))`, which is unscoped to the finalizing batch; an external account can donate `toToken` underlying to the batcher and inflate the value passed to `toToken().wrap(...)`, where wrapper supply limits, the `SafeCast.toUint64` cast on the exchange rate, and the post-wrap `InvalidExchangeRate` require can all revert. The `swappedAmount < toToken().rate()` edge case rounds `wrappedAmount` to zero and fails the same require. Independently, the callback wraps `IERC7984ERC20Wrapper.finalizeUnwrap` in a bare `try/catch` that re-validates the decryption proof and continues; if `finalizeUnwrap` reverted because the underlying transfer failed (paused stablecoin, blacklisted batcher, recipient restriction), the `fromToken` underlying never arrived in the batcher and the `Cancel branch's rewrap` reverts on insufficient balance.

Liveness failures produce the same stuck `Dispatched` state without any step reverting. If the KMS gateway is unavailable or `_executeRoute` keeps returning `ExecuteOutcome.Partial`, the callback never reaches a terminal branch. The contract's own warning above `_executeRoute` acknowledges this directly: "Failure to do so results in user deposits being locked indefinitely." `dispatchBatch` is also permissionless, so an actor with a small or zero deposit can advance the current batch into `Dispatched` and shrink the window that depositors have to quit.

Consider adding a permissionless, timeout-based cancellation path that moves a batch out of `Dispatched` once the elapsed time since dispatch exceeds a deployer-configurable threshold, routing the timeout transition to `Canceled` so depositors recover via `quit`. Consider also verifying unwrap receipt past the `finalizeUnwrap` catch branch (via balance delta or via `unwrapRequester(unwrapRequestId_)` returning `address(0)`), computing the `Complete` branch's `swappedAmount` from a pre/post balance delta around `_executeRoute` rather than the batcher's global balance, and handling the `swappedAmount < toToken().rate()` zero-rate case explicitly. Consider gating `dispatchBatch` behind a virtual `_authorizeDispatch()` predicate, exposing a sweep for unrelated `toToken` underlying donations to the batcher, and documenting the liveness assumptions on the KMS gateway, the underlying ERC-20, and `_executeRoute`.

Update: Acknowledged, not resolved. The OpenZeppelin development team stated:

As documented, the integrator must choose wrapper contracts that are correctly configured to ensure wrapping will not revert. Beyond this, the integrator chooses when and why a user can cancel the batch by deciding what to return from the `_executeRoute` function. We are unable to force an cancelation flow since that would potentially require clawing back spent funds somehow.

M-03 ERC-2771 Meta-Transactions Make the Forwarder the Ciphertext Principal in ERC7984Rwa

The codebase mixes two principal-extraction conventions. Role checks in `ERC7984Rwa` use the `onlyAdmin` and `onlyAgent` modifiers, which delegate to `_checkRole(role)` and internally resolve to `_checkRole(role, _msgSender())`. FHE ACL operations, by contrast, use raw `msg.sender`: `FHE.allow(mintedAmount, msg.sender)` to grant persistent ACL on a result and `FHE.isAllowed(encryptedAmount, msg.sender)` to gate operations requiring prior ACL on an input. The two methods are equivalent in a normal transaction context but diverge when a derived contract integrates `ERC2771Context`: `_msgSender()` returns the meta-transaction signer, while `msg.sender` remains the trusted forwarder.

Assuming ERC-2771 support is intended, this mismatch is a real bug. The fix is not a blanket `msg.sender` substitution, however, because the codebase contains three semantically distinct uses of `msg.sender`:

1. **User and signer principal.** FHE ACL grants and checks (`FHE.allow`, `FHE.isAllowed`, `FHE.allowTransient`) and the operator gate at `isOperator(from, msg.sender)` intend the user. Functions that grant ACL via `FHE.allow(..., msg.sender)`, such as `confidentialMint`, `confidentialBurn`, `recoverAddress`, and `forceConfidentialTransferFrom`, leak persistent decryption authority to the forwarder under ERC-2771. Functions that require ACL via `FHE.isAllowed(..., msg.sender)`, such as the raw-`uint64` privileged variants and `requestDiscloseEncryptedAmount`, revert because the signing agent's ACL grants are on itself, not on the forwarder. These sites need migration to `_msgSender()`.
2. **Contract identity and immediate-caller checks.** The hook-membership gate at `_modules.contains(msg.sender)` and the expected-token check at `msg.sender == address(fromToken())` verify the calling contract's address. These must remain raw `msg.sender`.
3. **Library-bound caller binding.** `FHE.fromExternal(encryptedAmount, inputProof)` in `@fhevm/solidity` validates the input proof against the contract-layer `msg.sender`. No contract-side override exists. Under ERC-2771, meta-transaction users must generate their input proof bound to the forwarder address rather than to their own, or wait for the library to expose a meta-transaction-aware proof check; the contract-layer migration alone does not complete ERC-2771 support.

Consider migrating the user-principal FHE ACL sites to `_msgSender()`, ideally via a single overridable virtual `_aclSender()` so subclasses opting out of meta-transaction support can override back to `msg.sender`. Consider preserving raw `msg.sender` at the contract-identity sites since those verify the calling contract rather than the user. Consider also documenting the `FHE.fromExternal` proof-binding constraint in the NatSpec and integration guide so deployers enabling ERC-2771 understand that input proofs must be generated for the forwarder address.

Update: Resolved in [pull request #382](#) at commit [3960a01](#). The OpenZeppelin development team stated:

We decided to fully block meta transactions here. The only contract that currently inherits `Context` is `ERC7984Rwa`, and we overrode the context functions to without the virtual modifier.

M-04 Concurrent Partial Batches Can Misallocate Target-Underlying Output Across Batches

`BatcherConfidential` allows multiple batches to be in the `Dispatched` state simultaneously because `dispatchBatch` only advances `_currentBatchId` and does not wait for prior batches to finalize, and it permits `_executeRoute` to return `ExecuteOutcome.Partial` across callbacks until completion. When a batch completes, the `Complete` branch of `dispatchBatchCallback` computes its output using `IERC20(toToken().underlying()).balanceOf(address(this))`, the batcher's full target-underlying balance rather than a per-batch delta.

If batch `A` returns `Partial` and leaves intermediate `toToken` underlying in the batcher, and batch `B` subsequently completes, the `Complete` branch for `B` reads `swappedAmount = balanceOf(this)`, capturing both its own output and `A`'s intermediate output. Batch `B`'s exchange rate is inflated, and batch `A` later finalizes with reduced output. Value is silently shifted from `A`'s depositors to `B`'s depositors with no revert and no clear event signal. Exploitation does not require an attacker; ordinary concurrent operation is sufficient. However, a depositor in the completing batch can time their callback deliberately to maximize the shifted value by joining a batch positioned to complete just after another batch's `Partial` step deposits intermediate output.

The contract documentation in the `ExecuteOutcome.Partial` enum and the `_executeRoute` NatSpec states that intermediate `Partial` steps must not result in

`toToken` underlying being transferred into the batcher. This invariant is not enforced on chain, and it is easy to violate in natural multi-step route implementations: any swap or bridge step that produces the target underlying will deposit it into the batcher (the typical recipient of swap output) between intermediate steps. A deployer who writes a straightforward multi-step route can therefore silently introduce the misallocation.

Consider tracking per-batch output using pre/post balance deltas around `_executeRoute`, snapshotting `balanceOf(address(this))` before the route call and using the post-call increment as `swappedAmount`. For multi-step `Partial` routes, consider storing the per-batch accumulation in the batch struct and updating it on every `Partial` return so the final `Complete` call wraps only the current batch's contribution. Alternatively, consider disallowing concurrent `Dispatched` batches or rejecting `Partial` routes that produce target underlying before completion. Consider also elevating the intermediate-output constraint from inline documentation to an explicit invariant in the `_executeRoute` NatSpec and the deployer integration guide, with a recommended compliant-route pattern, so the failure mode is understood at integration time rather than discovered through depositor losses.

Update: Resolved in [pull request #385](#) at commit [0097ca9](#). The OpenZeppelin development team stated:

Partial steps are now blocked from changing the ERC-20 balance of underlying `toToken`.

M-05 Operator-Held Transient Access Can Be Escalated to Permanent Public Disclosure of Transferred Amounts

The operator-pattern entrypoints `confidentialTransferFrom` and `confidentialTransferFromAndCall` admit any caller authorized via `isOperator(from, msg.sender)` and return an `uint64 transferred` handle representing the amount actually moved. Both paths grant the operator transient ACL on `transferred` via `FHE.allowTransient(transferred, msg.sender)` (and the same grant in `_transferAndCall`), intended for same-transaction processing of the encrypted amount by the operator.

`requestDiscloseEncryptedAmount` gates only on `FHE.isAllowed(encryptedAmount, msg.sender)` and, when the check passes, calls `FHE.makePubliclyDecryptable(encryptedAmount)`. Transient ACL satisfies `FHE.isAllowed` during the transaction in which it was granted, and

`makePubliclyDecryptable` is a permanent marker on the handle. In a single transaction the operator can therefore chain `confidentialTransferFrom(...)` to obtain `transferred`, then `requestDiscloseEncryptedAmount(transferred)` to permanently mark the handle as publicly decryptable. The `AmountDiscloseRequested` event surfaces the request, but the on-chain state change is the permanent decryptability flag, not the event itself.

Once the handle is publicly decryptable, `discloseEncryptedAmount` is permissionless: any caller can submit a valid KMS-signed decryption proof and trigger emission of the cleartext via the `AmountDisclosed` event. The transient access that the operator-pattern entrypoints grant for in-transaction processing is thereby escalated into permanent, publicly emittable cleartext exposure of the amount moved. The holder authorizing the operator authorized transfers on their behalf, not amount disclosure; the privilege escalation lies in the gap between those two scopes. For the raw-`euint64` variant of `confidentialTransferFrom`, the operator may not even hold prior knowledge of the cleartext (the holder previously granted them ACL on the handle for transfer purposes), so the escalation can leak amounts the operator did not originally know.

Consider gating `requestDiscloseEncryptedAmount` on a stronger authorization signal than `FHE.isAllowed`. One option is to restrict the check to persistent ACL grants only, filtering out transient access so operator-held transient grants cannot satisfy the gate. Another is to require an explicit holder authorization (a signature, a per-handle disclosure-request whitelist, or a role-based predicate) before `FHE.makePubliclyDecryptable` is called, so that disclosure intent is distinguished from transfer authority. Consider also documenting in the NatSpec that transient handle access is not sufficient to request public disclosure and that operators authorized for transfers do not inherit disclosure rights.

Update: Acknowledged, not resolved. The OpenZeppelin development team stated:

Note that all approvals are considered equal by the ACL. A transient approval recipient can freely give out persistent approvals (to themselves as well). Documentation was added, noting that operators have ACL access for transfer amounts in transfers they initiate.

Notes & Additional Information

N-01 `confidentialAvailable` Grants Persistent ACL Without Validating Result or Caller

`ERC7984Freezable.confidentialAvailable` is `public` and unconditionally calls `FHE.allowThis(amount)` and `FHE.allow(amount, account)` on the result of `_confidentialAvailable`, which itself reduces to `FHESafeMath.saturatingSub` over `confidentialBalanceOf(account)` and `confidentialFrozen(account)`. The function validates neither the result handle's initialization status nor the caller's relationship to `account` before issuing the persistent grants.

The audited changes replace the prior `tryDecrease + FHE.select(success, unfrozen, FHE.asEuint64(0))` pattern with `FHESafeMath.saturatingSub`. The previous implementation always returned an initialized zero because `FHE.select` produces an FHE-computed handle. The new helper may return an uninitialized handle when both inputs are uninitialized, per the library's [documented contract](#). For an account with no balance and no frozen amount, `_confidentialAvailable` therefore returns an uninitialized handle, and the unconditional `FHE.allow` on it depends on undocumented runtime semantics: if the runtime reverts, this is a behavioral regression against the prior implementation; if it silently no-ops, the contract relies on undocumented library behavior. The codebase's own `FHESafeMathMock.saturatingSub` guards the comparable code with `if (FHE.isInitialized(result))`.

The function is also callable by any third party with any `account` argument. Each invocation produces a fresh ciphertext handle through `saturatingSub` and registers a persistent ACL entry for `account` on that handle, accumulating persistent ACL state on the FHE runtime side with no caller-authorization check. The NatSpec describes the persistent grant but does not signal that it is reachable from arbitrary callers. The mock at `FHESafeMathMock.saturatingSub` incidentally illustrates the tighter pattern by granting ACL to `msg.sender` rather than to a separately supplied `account`.

Consider guarding the ACL grants with `if (FHE.isInitialized(amount))` so uninitialized handles do not produce undefined runtime behavior. Consider restricting the persistent grant to cases where the caller has a legitimate relationship to `account` (such as

`account == msg.sender`), or splitting the function into a transient-grant variant callable by anyone and a persistent-grant variant restricted to the named account. If preserving the current third-party-callable shape is deliberate (for example to support regulator-style queries), consider documenting the design choice in the natspec so deployers understand the persistent-ACL footprint and the absence of caller authorization.

Update: Acknowledged, not resolved. The OpenZeppelin development team stated:

`saturatingSub` always returns an initialized value. We prefer to keep `confidentialAvailable` as is — it is essentially a getter that can be called by the frontend, then presented to the user seamlessly. Given that tx fees are called by `tx.origin` we aren't concerned about the storage bloat.

N-02 Unlimited Underlying Approvals Expose Batchers-Held Funds to Wrapper Compromise

`BatcherConfidential` grants each wrapper unlimited approval over its corresponding underlying ERC-20 during construction, calling `SafeERC20.forceApprove(..., type(uint256).max)` once for the `fromToken` underlying and once for the `toToken` underlying. The approvals are never reduced after construction. The standing allowance enables `fromToken().wrap(...)` in the cancel branch and `toToken().wrap(...)` in the complete branch to pull underlying from the batcher across batch lifetimes without per-call approval.

The canonical `ERC7984ERC20Wrapper` does not call `transferFrom(batcher, ...)` against the standing approval from any of its public surfaces (`wrap`, `unwrap`, `finalizeUnwrap`), so the unlimited approval is not directly exploitable in a canonical deployment. The audit's trust model already treats the wrapper as a structurally trusted dependency, and a compromised wrapper has many attack vectors beyond approval-based drains.

The defense-in-depth concern is for deployers extending the wrapper with upgradeability. If a deployer composes the wrapper with an upgrade mechanism, the standing approval becomes a persistent capability that a future implementation can exercise to drain all batcher-held underlying balance via a single `transferFrom`. The blast radius is the entire batcher-held balance of both underlyings, across all batches' deposits, intermediate `Partial` outputs, and donations.

Consider replacing the construction-time unlimited approvals with bounded approvals around each `wrap` call (`forceApprove(amount) -> wrap -> forceApprove(0)`), aligning

with the modern OpenZeppelin approval-handling convention for high-value standing allowances. Consider also documenting the approval blast radius in the wrapper-integration section of the deployer guide so deployers extending the wrapper with upgradeability understand the standing-allowance surface they inherit.

Update: Acknowledged, not resolved. The OpenZeppelin development team stated:

The batcher is tightly coupled to the wrapper, and there is no route to avoid the wrapper. This is an inherent part of the architecture. Reducing approvals would increase costs without improving the security.

N-03 `onUninstall` Lacks Installed-Module Privileges and Reverts Silently

`ERC7984Hooked._uninstallModule` removes the module from the `_modules` membership set before invoking `IERC7984HookModule.onUninstall`, and dispatches the callback via `LowLevelCall.callNoReturn`, which swallows any revert from the callee. The combination produces two subtle properties that module authors may not anticipate.

First, the callback executes after the module has lost installed-module trust. While `onUninstall` runs, `_validateHandleAllowance` no longer admits the module as a privileged caller because `_modules.contains(msg.sender)` is already false. Cleanup logic that depends on the installed-module ACL grant path, for example a deinit step that issues transient ACL on a token handle so the module can read its own per-token state during cleanup, will fail. The base `ERC7984HookModule.onUninstall` does not rely on this access, but a deployer-extended module that does will silently misbehave.

Second, the `callNoReturn` dispatch swallows reverts. If `onUninstall` reverts for any reason, the token-side state still records the module as uninstalled (the membership removal has already taken effect and `ERC7984HookedModuleUninstalled` is emitted), but the module-side state may remain inconsistent. The module's own `_installed[token]` flag may stay set, and any external bookkeeping the module maintains may diverge from the token's view, with no on-chain signal that the cleanup failed.

Consider documenting both properties on the `IERC7984HookModule.onUninstall` interface and on the `_uninstallModule` NatSpec: cleanup logic must not rely on installed-module privileges (those are revoked before the callback runs), and cleanup failures are silently absorbed and do not abort the token-level uninstall. Consider also providing a recommended `onUninstall` template in the integration guide that performs cleanup without requiring

privileged token access, so deployers extending the example modules have a known-good pattern.

Update: Resolved in [pull request #380](#) at commit [8492b03](#). The OpenZeppelin development team stated:

| The `onUninstall` hook has been removed.

N-04 Hardcoded Selector Literals in `_isForceTransfer` Are Silent-Failure-Prone

`_isForceTransfer` compares `msg.sig` against two hardcoded 4-byte literals plus the compile-time `this.recoverAddress.selector`: `0x6c9c3c85` with a comment naming `forceConfidentialTransferFrom(address,address,bytes32,bytes)`, `0x44fd6e40` with a comment naming `forceConfidentialTransferFrom(address,address,bytes32)`, and `this.recoverAddress.selector`, which is derived at compile time and therefore resistant to drift.

If a future change renames either `forceConfidentialTransferFrom` overload, reorders its parameters, or alters a parameter type (for example through an [@fhevm/solidity](#) update that changes how `externalEuint64` ABI-encodes), the accompanying comments will be silently incorrect and the literals will no longer match the actual selectors. The Solidity compiler does not warn on this drift. The downstream effect is that force-transfer calls would no longer bypass the pause modifier and sender restriction check at the `_update` entry point, so force transfers would revert with `EnforcedPause` or with the corresponding sender-restriction error from `ERC7984Restricted` instead of executing.

Consider deriving both literal selectors at compile time using overload-aware function references on the `IERC7984Rwa` interface, the same pattern that already keeps `recoverAddress` drift-resistant. If the Solidity syntax for disambiguating the two `forceConfidentialTransferFrom` overloads proves awkward, consider adding a CI-time selector-assertion test that hashes the canonical signatures with `keccak256` and asserts equality against the hardcoded literals. Either approach catches the divergence at build time rather than at deployment.

Update: Resolved. The OpenZeppelin development team stated:

| The current test suite verifies that functions that rely on the `_isForceTransfer` function operate properly. Any drift in the selector would be detected there.

N-05 Code Quality and Documentation

Throughout the codebase, multiple opportunities for improving code quality and documentation were identified. For instance:

1. The class docstring on `ERC7984Rwa` overstates the force-transfer bypass scope. The docstring states that force-transfer operations bypass "pause and restriction checks", but in code only the sender-side restriction and the pause modifier are bypassed via `_isForceTransfer(msg.sig)`; the recipient-side check in `ERC7984Restricted._update` is preserved, so `forceConfidentialTransferFrom` reverts when the recipient is `BLOCKED` and `recoverAddress` reverts when the new account is `BLOCKED`. The asymmetric behavior is defensible but the docstring should reflect it.
2. Test coverage for `FHESafeMath.saturatingSub` on uninitialized operands is missing. The `test table for saturatingSub` does not include cases where one or both operands are uninitialized, even though `ERC7984Freezable._confidentialAvailable` now relies on `saturatingSub` for fresh accounts where both balance and frozen handles may be uninitialized. Cases `[undefined, undefined, 0]`, `[undefined, 5, 0]`, and `[5, undefined, 5]` should be added to the `saturatingSubArgsOptions` table, locking the equivalence between the new `saturatingSub` path and the prior `tryDecrease + FHE.select` pattern as a regression-checked invariant.
3. Regression tests for the Multicall plus pause plus force-bypass composition are missing. The `ERC7984Rwa test suite` contains no `multicall` cases. The selector-based force-bypass relies on `msg.sig` resolving to the inner function selector inside a `delegatecall`, which holds under the OpenZeppelin v5.6 `Multicall` implementation pinned in `.gitmodules`. A future Multicall upgrade or replacement that mutates `msg.data` before the inner call would silently break the force-bypass logic. Four regression tests batching `pause() + forceConfidentialTransferFrom`, `pause() + recoverAddress`, `pause() + confidentialTransfer`, and `unpause() + confidentialTransfer` would lock the composition behavior in.
4. Unused import in `BatcherConfidential`. `BatcherConfidential.sol` imports the `uint128` type from `@fhevm/solidity/lib/FHE.sol`, but the contract operates exclusively on `uint64` and `uint64`.

Consider addressing all improvement recommendations mentioned above.

Update: Resolved at commit [ccb4572](#), [5193a0e](#), [7ad6d5b](#) and at commit [551c9b7](#). The OpenZeppelin development team stated:

| All points were addressed.

N-06 Installed Hook Modules Can Make Unbounded FHE ACL Grants That Survive Uninstall

The [ERC7984Hooked](#) extension allows installed modules to participate in the token's handle-access logic by routing through [_validateHandleAllowance](#). On this path, an installed module can invoke the FHE ACL grant primitives via the token with no contract-side bound on which handle is targeted, which account receives the grant, or whether the grant is transient or persistent.

The handle dimension is unbounded: a module can request ACL on handles that belong to arbitrary holders' private state, which makes any installed module implicitly trusted to access every encrypted balance, deposit, or allowance the token manages. The account and persistence dimension is also unbounded: a module can request persistent grants to arbitrary third-party accounts, even though the helper [ERC7984HookModule._getTokenHandleAllowance](#) follows the `account == msg.sender && persistent == false` pattern as an honest-module convention. The token does not enforce that pattern.

The combined effect is that any installed module, including a third-party module installed by the deployer that contains a bug or a subtly malicious branch, can issue a persistent FHE ACL grant on any holder's encrypted state to any third-party address. The FHE ACL primitives expose no revocation interface accessible from the token, so the grant cannot be reversed after the fact and is not undone by uninstalling the module via [_uninstallModule](#). A single compromised or buggy module is sufficient to permanently breach the confidentiality property of every holder's state, with no on-chain remediation available.

Consider enforcing the account and persistence bound on the token side in [_validateHandleAllowance](#) and any other module-callable handle-access entry point, requiring `account == msg.sender` and `persistent == false` whenever `msg.sender` is an installed module. This lifts the existing helper convention to a token-side guarantee and structurally closes the revocation gap, since no module can produce a persistent grant that would need to be revoked.

Consider also documenting the residual trust assumption that installed modules retain the ability to access any handle the token manages, prominently in the `ERC7984Hooked` NatSpec and in the trust-model section of the deployer guide. Consider additionally recommending an admin-tier authority pattern (multi-sig or timelock) for module installation, and documenting that the deployer's choice of installed modules permanently shapes the token's confidentiality surface.

Update: Acknowledged, not resolved. The OpenZeppelin development team stated:

This is an inherent aspect of the design of the hook module system. We've expanded the current documentation to describe the privacy trust in more detail.

Client Reported

CR-01 Trivial-Case Early Return in `ERC7984HolderCapHookModule._preTransfer` Skips the `super` Call

The hook-module framework in `ERC7984Hooked` chains `_preTransfer` overrides via `super` so that derived modules can compose additional compliance checks on top of an existing module. The base `ERC7984HookModule._preTransfer` is currently a no-op that returns `FHE.asEbool(true)`, but the chaining convention is what enables a custom subclass of an example hook module to inject its own checks while preserving the parent's behavior.

The implementation of `ERC7984HolderCapHookModule._preTransfer` violates this convention. When the call is for a mint (`to == address(0)`) or a self-transfer (`to == from`), the function takes the `trivial-case branch` and returns `FHE.asEbool(true)` directly, without calling `super._preTransfer`. The non-trivial branch correctly chains via `FHE.and(compliant, super._preTransfer(...))`. As a result, half of the call paths skip the `super` chain.

In isolation today the functional impact is bounded, because the base `_preTransfer` returns `true` and `FHE.and(true, true) == true`. The real impact arises in composition: any subclass that extends `ERC7984HolderCapHookModule` and threads its own compliance check through `super._preTransfer` finds those checks silently bypassed on every mint and every self-transfer. The sibling module

`ERC7984BalanceCapHookModule._preTransfer` does not have this issue because its `super` call is outside the `if/else` and fires in both branches.

Consider restructuring `ERC7984HolderCapHookModule._preTransfer` so that `super._preTransfer` is invoked unconditionally as the first statement, and the module's own compliance check is `FHE.and`-combined with the super result only in the non-trivial branch. Consider also applying the same pattern uniformly across all example hook modules so the super-chaining convention is explicit and harder to break in future modules.

Update: Resolved in [pull request #369](#) at commit [1da811c](#). The OpenZeppelin development team stated:

| *The issue has been fixed.*

Conclusion

The audited codebase introduces the `ERC7984Hooked` extension and its supporting module framework (a new `IERC7984HookModule` interface, the abstract `ERC7984HookModule` base, and two example modules implementing per-investor balance caps and per-token holder caps), a new identity-verification extension (`ERC7984IdentityCheck`), an agent-only account-recovery operation on `ERC7984Rwa`, a behavioral change in the core `ERC7984._update` removing the `ERC7984ZeroBalance` revert, two new `FHESafeMath` saturating primitives, and a constructor invariant on `BatcherConfidential` rejecting wrappers that share an underlying token. The diff reflects a substantial expansion of the library's compliance and composability surface rather than incremental refinements alone.

The audit identified one critical-severity issue concerning the hook-module framework: installed modules can grant persistent FHE ACL on arbitrary handles to arbitrary third-party accounts, and those grants survive uninstall because the FHE library exposes no revocation primitive for direct `allow(handle, account)` grants. A single buggy or subtly malicious module is therefore sufficient to permanently breach the confidentiality property of every holder's state, with no on-chain remediation available. The recommended fix is to enforce `account == msg.sender` and `persistent == false` on the token side whenever the caller is an installed module, which structurally prevents persistent grants from being made at all and closes the revocation gap as a side effect.

The five medium-severity findings concern distinct surfaces: a permissionless zero-amount transfer shortcut admitted by the `FHE.fromExternal` empty-proof path; a structural denial-of-service gap in `BatcherConfidential` where the `Dispatched` state has no time-bounded recovery path under finalization or cancellation reverts (consolidating donation-driven wrap reverts, the swallowed `finalizeUnwrap` catch branch, the `swappedAmount < rate()` zero-rate edge, Cancel-path rewrap failures, and liveness failures of the KMS gateway and `_executeRoute`); an ERC-2771 meta-transaction composition hazard where role checks and FHE ACL operations resolve to different principals when the contracts are composed with `ERC2771Context`; a value-allocation bug where concurrent `Partial` batches can misallocate target-underlying output across batches via the full-balance read in the `Complete` branch; and a privilege-escalation gap in the operator pattern where transient FHE ACL granted by `confidentialTransferFrom*` satisfies the `FHE.isAllowed` gate on `requestDiscloseEncryptedAmount`, letting an operator chain transfer authority into permanent public disclosure of the moved amount within a single transaction.

The five notes and additional information findings cover an `ACL`-hygiene observation on `confidentialAvailable` covering both result-handle validation and caller-relationship checks, a standing-approval blast-radius observation on `BatcherConfidential` for deployers extending the wrapper with upgradeability, a documentation-hardening note on the `onUninstall` lifecycle, hardcoded selector literals at risk of silent failure if signatures drift, and a code-quality and documentation bundle (docstring accuracy, missing test cases, and an unused import).

The codebase is well-structured and builds carefully on established OpenZeppelin patterns. The hook-module framework is the most security-sensitive addition and would benefit from explicit documentation of the trust model it imposes on installed modules, in particular the persistent-ACL grant surface and the asymmetric ordering between the install and uninstall callbacks. The `BatcherConfidential` state machine would benefit from a time-bounded escape path out of `Dispatched`, per-batch delta tracking that decouples batch finalization from the contract's global underlying balance, and an `_authorizeDispatch` predicate that lets deployers gate the dispatch trigger.

The OpenZeppelin Zama development team is greatly appreciated for their responsiveness and collaborative approach throughout the audit.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.